

The Six Grounded Hypercalculi: An Integrated Framework for Relativized Oracles, Metasystem Reflection, and Hypercomputation

Tyler Roost

Preprint, version 0.2.0 — September 18, 2026

Abstract

We introduce a unified mathematical and computational framework consisting of six mutually grounded calculi: Language Calculus ($\mathcal{C}_{\text{lang}}$), Meta Calculus ($\mathcal{C}_{\text{meta}}$), Hyper Calculus ($\mathcal{C}_{\text{hyp}}$), Ordinal Calculus ($\mathcal{C}_{\text{ord}}$), Real Analysis Calculus ($\mathcal{C}_{\text{real}}$), and Verification / Oracle Calculus ($\mathcal{C}_{\text{ver}}$). The central impetus of this architecture is to discover routes toward definably complete representations capable of handling hypercomputation-like oracle classes expected of superintelligences. Rather than positing unstratified magical computation or ungrounded global truth predicates, each calculus formalizes an explicit boundary between deductive computation and external oracle interaction. We formalize the ω -tower of Turing degrees, Turchin-style metasystem transitions governing cognitive reflection, transfinite derivation limits, well-founded ordinal termination bounds, continuous metric completions, and strict proof verifier soundness. The reference implementation states each calculus’s capability boundary explicitly, including the points at which a component returns a deterministic placeholder rather than a decision. Receipt emission under the Verifier Standard (VSTD) discipline is specified here as the intended verification interface for $\mathcal{C}_{\text{ver}}$; it is not implemented, and no result in this paper rests on it.

Keywords: Hypercalculi; Language Calculus; Meta Calculus; Hyper Calculus; Ordinal Calculus; Real Analysis Calculus; Verification and Oracle Calculus; Turing jumps; metasystem transition; superintelligence oracles; Verifier Standard.

1 Introduction and Foundational Impetus

A core theoretical requirement for modeling superintelligent agents is characterizing the interaction between finite deductive reasoning and higher-order cognitive oracles. In classical computability theory, Turing introduced oracle machines to formalize computation relative to an uncomputable set $A \subseteq \mathbb{N}$ [1]. Modern theoretical computer science extends this through the arithmetical hierarchy, the hyperarithmetic hierarchy Δ_1^1 , and transfinite Turing jump towers $\mathbf{0}^{(\alpha)}$ [4].

However, existing formal treatments typically segregate these phenomena: recursion theorists study abstract degree structures, logicians study proof-theoretic reflection, and computer scientists study programming language calculi. When building autonomous cognitive systems or verifiable foundation models, these components must interface seamlessly within an executable, refutable software architecture.

The Six Grounded Hypercalculi provide this integrated framework:

1. **Language Calculus ($\mathcal{C}_{\text{lang}}$):** Syntactic Abstract Syntax Tree (AST) manipulation, quotation typing, and a proof-checking fragment of Metamath.

2. **Meta Calculus** ($\mathcal{C}_{\text{meta}}$): Term rewrite systems, trajectory bisimulation, and Turchin meta-system transitions.
3. **Hyper Calculus** ($\mathcal{C}_{\text{hyp}}$): Transfinite limit evaluation, derivation operators, and permutation group actions.
4. **Ordinal Calculus** ($\mathcal{C}_{\text{ord}}$): Cantor normal form ordinal arithmetic below ω^ω , difference operators, and Veblen hierarchies.
5. **Real Analysis Calculus** ($\mathcal{C}_{\text{real}}$): Computable real numbers, Cauchy sequences, Dedekind cuts, interval arithmetic, and Surreal games.
6. **Verification and Oracle Calculus** ($\mathcal{C}_{\text{ver}} / \mathcal{C}_{\text{orc}}$): Relativized Turing degree queries, conclusion analysis, hyperoracles, leapfrog computations, and strict proof-checker soundness.

2 Language Calculus: Syntax, Quotation, and Database Scoping

The Language Calculus ($\mathcal{C}_{\text{lang}}$) governs the syntactic substrate upon which all symbolic reasoning takes place.

Definition 2.1 (Grammar and Abstract Syntax Tree Structure). An alphabet Σ is a set of constant and variable symbols. A formal grammar $\Gamma = (\Sigma, V, P, S)$ defines the language of well-formed expressions. An Abstract Syntax Tree (AST) represents parsed terms, where term quotation $\lceil t \rceil$ reifies syntactic objects into first-class data terms.

Capture-avoiding substitution $\text{sub}(e, x, v)$, under which no free variable of v becomes bound, and staged boundaries between object language and meta-language are part of this formal account.

In our reference implementation (`language_calculus.py`), $\mathcal{C}_{\text{lang}}$ realizes a strict subset of that account:

1. **Metamath Fragment Verification:** A database distinguishing constant declarations, variable typing hypotheses, essential assertion hypotheses, axioms and theorems, over which Reverse-Polish-Notation proofs are checked.
2. **Quotation:** `QuotationNode` reifies a syntactic object into a first-class data term.
3. **Adversarial Soundness:** The defects that would let the checker certify $\vdash \perp$ are pinned by `tests/test_proof_verifier_soundness.py`.

The omissions are load-bearing and are stated here rather than glossed. The implementation has *no* substitution: hypotheses are matched literally, so an axiom stated over P applies only to arguments that literally mention P . It has no disjoint-variable conditions, no frame-derived mandatory hypothesis ordering, no compressed proofs, and no nested statement scoping — the $\$[\dots\$]$ scope form is not parsed at all. It is therefore incomplete against real Metamath databases, which is the direction a proof checker may err in; unsoundness is the direction it may not.

3 Meta Calculus: Rewriting, Trace Comparison, and Reflection

The Meta Calculus ($\mathcal{C}_{\text{meta}}$) models dynamic state transformations and cognitive reflection principles.

Definition 3.1 (Term Rewriting System). A rewrite system is a pair $\mathcal{R} = (S, \rightarrow)$ where S is a set of terms and $\rightarrow \subseteq S \times S$ is a transition relation defined by a set of rewrite rules $\{l_i \rightarrow r_i\}$.

Definition 3.2 (Trace Equality). Two state execution trajectories $\tau_1 = (s_0, \dots, s_m)$ and $\tau_2 = (t_0, \dots, t_n)$ are *trace-equal* (`trajectory_bisimilar`) when $m = n$ and $s_i = t_i$ for every $i \leq m$.

Despite its name, which predates the observation, this predicate is not bisimulation, and it is not invariant under differing reduction step counts. Bisimilarity is a property of the branching structure of a transition system; the predicate receives two sequences and no transition system, so no function of its inputs can decide it. A witness: under the system whose only rule is $ab \rightarrow ba$ the state ab halts after one step, while under the system with both $ab \rightarrow ba$ and $ba \rightarrow ab$ it runs forever, so the two states are not bisimilar — yet their one-step traces are both (ab, ba) and the predicate returns true. What trace equality does decide is worth stating: two deterministic traces of equal length agree pointwise. What it does not decide is whether the states they start from behave alike. It is sound only over the prefix it covers, and a trace truncated by a step budget covers less than it appears to.

Remark 3.3 (Metasystem Transition and Reflection, stated as a target). Let M_k be a cognitive system and $\text{Tr}(M_k)$ its execution trace. A metasystem M_{k+1} operating in Meta Calculus monitors, analyzes, and controls M_k . The design target is that the local reflection principle

$$\text{RFN}(M_k) = \{\text{Prov}_{M_k}(\ulcorner \varphi \urcorner) \rightarrow \varphi\}$$

be evaluated within M_{k+1} without importing a Gödelian diagonal into M_k , in the manner of Feferman’s transfinite progression schema [3].

This is recorded as a target and not as a theorem. No proof is given here, and `meta_calculus.py` implements no provability predicate, no reflection schema, and no soundness check, so nothing in the reference implementation bears on it. Feferman’s own results show that claims of this shape are delicate at the limit stages, and the surrounding construction does not settle one.

4 Hyper Calculus: Derivations, Groups, and Transfinite Limits

The Hyper Calculus ($\mathcal{C}_{\text{hyp}}$) extends discrete algebraic manipulation into higher-order derivation operators, continuous group actions, and limit evaluation.

Definition 4.1 (Derivation Operator). A *derivation operator* D on an algebra $\mathcal{A}$ is a linear mapping satisfying the Leibniz product rule:

$$D(a \cdot b) = D(a) \cdot b + a \cdot D(b).$$

In $\mathcal{C}_{\text{hyp}}$, iterated operators D^n and transfinite limit operators D^λ compute algebraic recurrence along fundamental sequences of limit ordinals.

Furthermore, $\mathcal{C}_{\text{hyp}}$ incorporates finite permutation groups (`PermutationGroup`) acting on state configurations. Separately, and over a different structure, `commutator_bracket` forms the Lie bracket

$$[D_1, D_2] = D_1 \circ D_2 - D_2 \circ D_1$$

of two derivation operators, measuring their failure to commute. The two constructions are distinct and should not be conflated: a permutation group is multiplicative and carries no subtraction, so the bracket above is not defined on it. Non-commutativity of group elements is witnessed instead by the group commutator $ABA^{-1}B^{-1}$.

5 Ordinal Calculus: Stage Semantics and Termination Metrics

The Ordinal Calculus ($\mathcal{C}_{\text{ord}}$) provides inductive stage bounds with which a caller may argue that an operational loop terminates within well-founded limits. The bounds are a representation offered to the caller; the library applies them to no loop and enforces termination on nothing.

Definition 5.1 (Ordinal Arithmetic and Difference Operator). For ordinals $\alpha, \beta < \omega^\omega$ represented in Cantor normal form:

$$\alpha = \omega^k a_k + \cdots + \omega a_1 + a_0,$$

$\mathcal{C}_{\text{ord}}$ computes exact non-commutative addition $+_o$, multiplication $\cdot_o$, and the discrete ordinal difference $\Delta(\alpha, \beta)$, measuring the excess rank between operational stages.

Definition 5.2 (Veblen Hierarchy and Normal Functions). The Veblen hierarchy is generated by transfinite fixed-point *enumeration*. Set $\varphi_0(\beta) = \omega^\beta$. For successor indices, $\varphi_{\alpha+1}$ enumerates in increasing order the fixed points of φ_α , so that

$$\varphi_{\alpha+1}(\beta) = \text{the } \beta\text{-th ordinal } \gamma \text{ with } \varphi_\alpha(\gamma) = \gamma,$$

and for limit λ , φ_λ enumerates $\bigcap_{\xi < \lambda} \text{Range}(\varphi_\xi)$ in increasing order. Each φ_α is a normal function, which is what makes the next row's enumeration well defined. The least fixed point $\varphi_1(0) = \varepsilon_0$ is the proof-theoretic ordinal of Peano arithmetic.

Writing $\varphi_{\alpha+1}(\beta) = \text{Fix}(\varphi_\alpha)$ would be a category error: $\text{Fix}(\varphi_\alpha)$ is the class of fixed points, and the argument β selects a member of it.

Implementation boundary. `BoundedOrdinal` represents ordinals strictly below ω^ω in Cantor normal form. `VeblenHierarchy` therefore evaluates only the φ_0 row at finite β and *refuses* every other argument rather than returning an approximation: $\varphi_0(\omega)$ is ω^ω itself, and $\varphi_1(0) = \varepsilon_0$ lies far above it. The ε_0 bound is a property of the definition above and not of any value this library can represent, and no termination argument in the implementation is discharged at ε_0 .

6 Real Analysis Calculus: Continuous Metrics and Game Embeddings

To bridge symbolic logic with physical environments and economic utilities, the Real Analysis Calculus ($\mathcal{C}_{\text{real}}$) implements verified continuous mathematics.

Definition 6.1 (Grounded Real Representations). A computable real number is formalized via:

1. **Dedekind Cuts:** Partitions (A, B) of $\mathbb{Q}$ defining real bounds.
2. **Cauchy Sequences:** Sequences of grounded rationals $(q_n) \in \mathbb{Q}$ satisfying $|q_n - q_m| < 2^{-k}$ for $n, m \geq N(k)$.
3. **Interval Arithmetic:** Representing uncertain quantities as bounding intervals $[a, b] \subset \mathbb{R}$, with the invariant $a \leq b$ enforced at construction and inclusion monotonicity as the intended property.

Implementation boundary. `Interval` stores its bounds as IEEE-754 doubles and its arithmetic rounds to nearest, not outward. Inclusion monotonicity therefore holds up to floating-point rounding and not absolutely: a result may fall outside the computed interval by up to a unit in the last place. These are working bounds, not a verified enclosure, and the word *verified* should not be read as a soundness guarantee here. An exact enclosure would require either directed rounding or rational bounds; `GroundedRational` already uses `Fraction` and is exact, so the exact path exists but is not the one `Interval` takes.

$\mathcal{C}_{\text{real}}$ also implements numerical differentiation, Riemann integration, and Conway Surreal game values $(\{L \mid R\})$, unifying continuous analytical flows with discrete game-theoretic optimization.

7 Verification and Oracle Calculus: Relativized Degrees and Proof Soundness

The Verification and Oracle Calculus ($\mathcal{C}_{\text{ver}}$ / $\mathcal{C}_{\text{orc}}$) formalizes the interface between deductive computation, proof checking, and external oracle interaction.

7.1 Turing Degree Towers and Hyperoracles

Definition 7.1 (Turing Jump Operator and Oracle Tower). Let $\mathbf{0}$ denote the degree of computable sets. The n -th jump is $\mathbf{0}^{(n+1)} = (\mathbf{0}^{(n)})'$. The ω -tower combines all finite jumps:

$$\mathbf{0}^{(\omega)} = \bigoplus_{n < \omega} \mathbf{0}^{(n)}.$$

Hyperoracles reside strictly above the arithmetic jump tower, providing Π_1^1 -complete truth queries (Kleene's $\mathcal{O}$).

Definition 7.2 (Conclusion Analysis). When an agent poses a self-referential or diagonal query Q of rank r , conclusion analysis lifts Q to rank $r + 1$, so that diagonal queries are posed strictly one tier above the domain of discourse.

Implementation boundary. This is the sharpest gap between the mathematics above and `oracle_calculus.py`, and it is stated plainly because a reader is entitled to know what the module decides. It decides one thing: a query of complexity rank r posed to an oracle at level $\ell \leq r$ returns `UNDECIDABLE_AT_TIER`, which is a checkable consequence of the tier discipline and holds exactly on that condition. When $\ell > r$, the module returns `HALTS` or `LOOPS` derived from a hash of the query's identifiers. That verdict is a deterministic placeholder and not a result: the query carries no program to ask about, so no function of its inputs could be a halting predicate, and renaming a query changes its verdict. It is reproducible, and reproducibility is not evidence. The same holds of `Hyperoracle` and of the status returned by `ConclusionAnalysis`; what conclusion analysis establishes is the rank lifting to $r + 1$, not the answer delivered there.

7.2 Proof Verifier Soundness

A verification engine that accepts invalid proofs is worthless. In $\mathcal{C}_{\text{ver}}$, proof checker soundness is verified against adversarial attacks:

1. **Hypothesis Binding:** Axioms and theorems cannot drop or ignore declared hypotheses; every essential hypothesis must be explicitly discharged by an authenticated witness.
2. **Proof Citation Integrity:** A cited theorem must have its own proof verified; citations cannot bypass proof elaboration.
3. **Modus Ponens Precision:** Implication splitting must strictly occur at the root logical connective (parenthesis depth zero).
4. **Acyclicity of Proof Citations:** Self-referential citation loops are strictly detected and refused.

8 Unified Architecture and Verifier Standard Interoperability

The six calculi form a coherent verification pipeline:

$$\mathcal{C}_{\text{lang}} \text{ (Syntax \& Quoting)} \longrightarrow \mathcal{C}_{\text{meta}} \text{ (Rewriting \& Reflection)} \longrightarrow \mathcal{C}_{\text{hyp}} \text{ (Limits)} \longrightarrow \mathcal{C}_{\text{ord}} \text{ (Termination)} \longrightarrow \mathcal{C}_{\text{real}} \text{ (Metrics)} \longrightarrow \mathcal{C}_{\text{ver}} \text{ (Verification \& Proofs)}$$

The Verifier Standard (VSTD) claim discipline is the intended verification interface for this pipeline. Under it, a result would be accompanied by an immutable, content-addressed receipt recording its exact mathematical coordinates, assumptions, trust roots, and falsification conditions.

Implementation boundary. No such receipt is emitted. The library contains no receipt machinery of any kind and declares no runtime dependencies, so nothing in it computes a canonical digest or writes a claim record. The claims in this paper rest on the module-level tests and on the capability boundary each calculus states for itself; none rests on receipt evidence, and none should be read as carrying any. Emitting VSTD receipts is future work — and the discipline it would impose, naming the scope, the limitations and the falsification condition of every result, is the discipline the boundary notes above are already written in.

9 Conclusion

The Six Grounded Hypercalculi assemble a modular and refutable computational framework for hypercomputational oracles, language stratification, metasystem reflection, transfinite dynamics, and verified proof checking. The framework is not complete, and the preceding sections mark where it stops: the Metamath fragment has no substitution, the ordinal representation cannot reach ε_0 , trace equality is not bisimulation, the reflection principle is a target rather than a theorem, and the oracle verdicts outside the tier rule are placeholders. Portable receipt generation under the Verifier Standard is the intended next step and is not a property of the current implementation. What the framework offers today is an enforced capability boundary per calculus and a refusal to extrapolate past it.

References

- [1] Alan M. Turing. Systems of Logic Based on Ordinals. *Proceedings of the London Mathematical Society* s2-45(1), 161–228, 1939. [doi:10.1112/plms/s2-45.1.161](https://doi.org/10.1112/plms/s2-45.1.161).
- [2] Valentin F. Turchin. *The Phenomenon of Science: A Cybernetic Approach to Human Evolution*. Columbia University Press, New York, 1977.
- [3] Solomon Feferman. Transfinite Recursive Progressions of Axiomatic Theories. *The Journal of Symbolic Logic* 27(3), 259–316, 1962.
- [4] Hartley Rogers, Jr. *Theory of Recursive Functions and Effective Computability*. McGraw-Hill, New York, 1967.
- [5] Tyler Roost. *Ordinal Arithmetic: An Ordinal-First Metalanguage Approach to Definable Arithmetic Truth*. Preprint, version 0.3.0, 2026.
- [6] Tyler Roost. *Constructive Quine Fixed-Point Synthesis, Quadrilateral Filtration, and APG Bisimulation*. Preprint, version 0.2.0, 2026.
- [7] Tyler Roost. *Constructive Grounded Hyperset Theory: Anti-Foundation, Bisimulation Quotients, and SCC Condensation*. Preprint, version 0.2.0, 2026.